Hướng dẫn tự xây dựng app Dịch vụ bằng Flutter

Sep 25, 2026 · @THIL

Tổng quan

Thiết kế có 2 màn hình: **Danh sách dịch vụ** và **Chi tiết dịch vụ**. Frame thứ 3 trong thiết kế vẫn là màn Chi tiết, chỉ là đã cuộn xuống nên nút "Đặt lịch" ẩn đi. Doc này hướng dẫn sếp tự viết lần lượt 10 bước, bước nào cũng chạy được ngay sau khi viết xong.

Cách dùng doc này

1. Mỗi bước có 3 phần: **Mục tiêu** (làm xong thì thấy gì), **Giải thích** (tại sao làm như vậy), **Code** (gõ lại, đừng copy nguyên, để nhớ lâu).
2. Làm xong mỗi bước thì chạy `flutter run` và bấm `r` (hot reload) để kiểm tra.
3. Bị kẹt thì đọc mục "Lỗi thường gặp" ở cuối doc, hoặc để comment ngay trong doc.

Phân tích thiết kế thành các thành phần

Màn hình	Thành phần	Widget chính	File
Danh sách	Header: nút back + tiêu đề "Dịch vụ"	`AppBar`	`service_list_page.dart`
Danh sách	Thanh danh mục cuộn ngang, mục đang chọn tô vàng	`ListView` ngang	`category_bar.dart`, `category_item.dart`
Danh sách	Lưới 2 cột: ảnh + tiêu đề tối đa 3 dòng	`GridView` / `SliverGrid`	`service_card.dart`
Chi tiết	Banner lớn + nút back tròn trắng đè lên ảnh	`Stack` + `Positioned`	`detail_header.dart`
Chi tiết	Card tiêu đề trắng nổi, đè một nửa lên banner	`Stack` + `Positioned` với `bottom` âm	`detail_header.dart`

Màn hình	Thành phần	Widget chính	File
Chi tiết	Nội dung: đoạn văn, bullet, ảnh, tiêu đề phụ	`sealed class` + `switch`	`content_block_view.dart`
Chi tiết	Nút "Đặt lịch" gradient, ẩn khi cuộn xuống	`AnimatedSlide` + `ScrollController`	`booking_button.dart`

Kiến trúc: feature-first

Code được chia theo **tính năng** (feature), không chia theo loại file. Mọi thứ của tính năng "dịch vụ" nằm chung trong `features/services/`. Khi app có thêm tính năng mới như `booking` hay `profile`, sếp chỉ cần tạo thêm thư mục ngang hàng, không phải lục qua nhiều nơi.

Mỗi feature có 3 tầng, dữ liệu chạy một chiều:

```
flowchart LR
  A[data<br/>models + repository] --> B[providers<br/>Riverpod state]
  B --> C[presentation<br/>pages + widgets]
  C -- user tap --> B
```

- **data**: định nghĩa dữ liệu trông như thế nào (model) và lấy dữ liệu ở đâu (repository: mock hoặc API).
- **providers**: giữ state, ví dụ danh mục đang chọn là gì, danh sách đang loading hay đã có dữ liệu.
- **presentation**: chỉ vẽ UI dựa trên state. Widget không gọi API trực tiếp.

Cây thư mục cuối cùng

```
lib/
├── main.dart
├── app.dart
├── core/
│   ├── theme/
│   │   ├── app_colors.dart
│   │   ├── app_text_styles.dart
│   │   └── app_theme.dart
│   └── router/
│       └── app_router.dart
└── features/
```

```
    └── services/
        ├── data/
        │   ├── models/
        │   │   ├── service_category.dart
        │   │   ├── service_item.dart
        │   │   └── content_block.dart
        │   ├── mock_data.dart
        │   └── service_repository.dart
        ├── providers/
        │   └── service_providers.dart
        └── presentation/
            ├── pages/
            │   ├── service_list_page.dart
            │   └── service_detail_page.dart
            └── widgets/
                ├── category_bar.dart
                ├── category_item.dart
                ├── service_card.dart
                ├── detail_header.dart
                ├── content_block_view.dart
                └── booking_button.dart
```

Packages sử dụng

Package	Dùng để	Tương đương bên web
`flutter_riverpod`	Quản lý state, cache dữ liệu async	Zustand + React Query
`go_router`	Điều hướng giữa các màn, URL có tham số	Next.js App Router
`cached_network_image`	Tải ảnh và cache, có placeholder	`next/image`

Lộ trình

Bước	Nội dung	Làm xong thấy gì
0	Chuẩn bị project	App chạy, đã cài xong packages
1	Theme	Có sẵn màu và font dùng chung
2	Models + mock data	Có dữ liệu giả để hiển thị
3	Providers	Có state cho danh mục, danh sách, chi tiết

Bước	Nội dung	Làm xong thấy gì
4	Router	Chuyển qua lại được giữa 2 màn
5	Màn danh sách + thanh danh mục	Header và thanh danh mục bấm chọn được
6	Card + lưới 2 cột	Màn 1 hoàn chỉnh
7	Header chi tiết	Banner + card tiêu đề đè lên
8	Nội dung chi tiết	Đoạn văn, bullet, ảnh
9	Nút Đặt lịch	Màn 2 hoàn chỉnh, nút ẩn khi cuộn

Bước 0: Chuẩn bị project

Mục tiêu: project `test_app` đã cài đủ packages, có sẵn các thư mục rỗng và vẫn chạy được app mặc định.

0.1 Thêm packages

Mở terminal ở thư mục project rồi chạy:

```
cd ~/Documents/flutter/test_app
flutter pub add flutter_riverpod go_router cached_network_image
```

`flutter pub add` tự chọn version mới nhất tương thích rồi ghi vào `pubspec.yaml`. Dùng lệnh này sẽ không bị lỗi thụt lề YAML như khi sửa tay. Chạy xong, mở `pubspec.yaml` ra xem, sẽ thấy 3 dòng mới trong `dependencies:`.

0.2 Tạo cấu trúc thư mục

```
mkdir -p lib/core/theme lib/core/router
mkdir -p lib/features/services/data/models
mkdir -p lib/features/services/providers
mkdir -p lib/features/services/presentation/pages
mkdir -p lib/features/services/presentation/widgets
```

0.3 Ảnh

Trong lúc học, mock data sẽ dùng ảnh online từ `picsum.photos` nên chưa cần thêm file ảnh nào. Khi có ảnh thật từ designer, ví dụ icon "Dịch vụ hot", sếp làm như sau:

1. Tạo thư mục `assets/images/` và bỏ ảnh vào.
2. Khai báo trong `pubspec.yaml`, thụt lề đúng 2 space dưới `flutter:`:

```
flutter:
  uses-material-design: true
  assets:
    - assets/images/
```

3. Trong code, dùng `Image.asset('assets/images/hot.png')`.

0.4 Kiểm tra

```
flutter pub get
flutter run -d "iPhone 18 Pro Max"
```

App đếm số mặc định hiện lên là được. Nếu muốn nhanh hơn, sếp có thể chạy trên Chrome bằng `-d chrome`. Tuy vậy, nên kiểm tra giao diện trên simulator iPhone vì thiết kế làm cho mobile.

Bước 1: Theme (màu sắc và chữ)

Mục tiêu: gom tất cả màu và kiểu chữ về một chỗ. Sau này sếp viết `AppColors.primary` thay vì `Color(0xFF2B3A9E)` rải rác khắp nơi.

Giải thích: Đây là ý tưởng giống design tokens trong Tailwind. Khi designer đổi màu xanh chủ đạo, sếp chỉ sửa 1 dòng thay vì 50 dòng. Màu dưới đây em ước lượng từ ảnh thiết kế. Nếu có file Figma, sếp lấy mã màu chính xác từ đó và thay vào.

1.1 `lib/core/theme/app_colors.dart`

```
import 'package:flutter/material.dart';

/// Design tokens: every color in the app comes from here.
abstract final class AppColors {
  // Brand
  static const Color primary = Color(0xFF2B3A9E); // title "Dich vu",
headings
  static const Color primaryLight = Color(0xFF3F5BD9); // gradient start
  static const Color primaryDark = Color(0xFF1E2A8A); // gradient end
  static const Color gold = Color(0xFFE8A33C); // selected category

  // Surfaces
  static const Color background = Color(0xFFF1F3FB); // light blue page
background
  static const Color surface = Colors.white; // cards

  // Text
```

```
  static const Color textPrimary = Color(0xFF1F1F1F);
  static const Color textSecondary = Color(0xFF4A4A4A);

  // Booking button gradient
  static const LinearGradient buttonGradient = LinearGradient(
    colors: [primaryLight, primaryDark],
    begin: Alignment.topCenter,
    end: Alignment.bottomCenter,
  );
}
```

- `abstract final class`: không tạo instance được và không kế thừa được. Class này chỉ dùng làm nơi chứa hằng số (cú pháp có từ Dart 3).
- `Color(0xFF2B3A9E)`: `FF` là độ mờ (alpha, 100%), phần sau là mã hex `#2B3A9E`.

1.2 `lib/core/theme/app_text_styles.dart`

Đọc thiết kế sẽ thấy có khoảng 6 kiểu chữ. Mỗi kiểu được đặt tên theo **vai trò** của nó trong giao diện, không đặt theo cỡ chữ:

```
import 'package:flutter/material.dart';

import 'app_colors.dart';

abstract final class AppTextStyles {
  static const TextStyle appBarTitle = TextStyle(
    fontSize: 22,
    fontWeight: FontWeight.w600,
    color: AppColors.primary,
  );

  static const TextStyle categoryLabel = TextStyle(
    fontSize: 12,
    height: 1.3,
    color: AppColors.textPrimary,
  );

  static const TextStyle cardTitle = TextStyle(
    fontSize: 14,
    height: 1.35,
    color: AppColors.textPrimary,
  );

  static const TextStyle detailTitle = TextStyle(
    fontSize: 17,
    height: 1.35,
```

```
    fontWeight: FontWeight.w600,
    color: AppColors.textPrimary,
  );

  static const TextStyle sectionHeading = TextStyle(
    fontSize: 17,
    fontWeight: FontWeight.w700,
    color: AppColors.textPrimary,
  );

  static const TextStyle body = TextStyle(
    fontSize: 15,
    height: 1.4,
    color: AppColors.textSecondary,
  );

  static const TextStyle button = TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.w500,
    color: Colors.white,
  );
}
```

- `height: 1.35` là khoảng cách dòng, bằng 1.35 lần cỡ chữ (giống `line-height` trong CSS). Tiêu đề 3 dòng trong card cần giá trị này để không bị dính chữ.
- Không khai báo `fontFamily` thì iOS tự dùng SF Pro, đúng với font trong thiết kế.

1.3 `lib/core/theme/app_theme.dart`

`ThemeData` là cấu hình mặc định cho toàn app. Ví dụ mọi `Scaffold` sẽ tự có nền xanh nhạt, mọi `AppBar` tự có nền trắng và tiêu đề ở giữa.

```
import 'package:flutter/material.dart';

import 'app_colors.dart';
import 'app_text_styles.dart';

abstract final class AppTheme {
  static ThemeData get light => ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: AppColors.primary),
        scaffoldBackgroundColor: AppColors.background,
        appBarTheme: const AppBarTheme(
          backgroundColor: AppColors.surface,
          surfaceTintColor: Colors.transparent, // avoid Material 3 tint on
scroll
          elevation: 0,
          centerTitle: true,
          titleTextStyle: AppTextStyles.appBarTitle,
          iconTheme: IconThemeData(color: AppColors.primary),
        ),
      );
}
```

Tự kiểm tra: bước này chưa hiển thị gì lên màn hình. Sếp chạy `flutter analyze`, không có lỗi là đạt.

Bước 2: Models, mock data và repository

Mục tiêu: định nghĩa dữ liệu mà 2 màn hình cần và tạo dữ liệu giả để làm UI trước, chưa cần backend.

Giải thích: Nhìn vào thiết kế và liệt kê xem có những "thứ" gì:

- **Danh mục** (`ServiceCategory`) gồm icon tròn và tên. Mục "Dịch vụ hot" đặc biệt vì có màu vàng.
- **Dịch vụ** (`ServiceItem`) gồm ảnh thumbnail, tiêu đề, banner và nội dung chi tiết.
- **Nội dung chi tiết** không phải một đoạn text duy nhất. Nó là danh sách các khối: đoạn văn, bullet, ảnh, tiêu đề phụ. Vì vậy em dùng `sealed class` để mô tả "một khối có thể là 1 trong 4 loại".

2.1 `data/models/service_category.dart`

```
class ServiceCategory {
  const ServiceCategory({
    required this.id,
    required this.name,
    required this.iconUrl,
    this.isHot = false,
  });

  final String id;
  final String name;
  final String iconUrl;
  final bool isHot; // "Dich vu hot" uses a special fire icon + gold color
}
```

Tất cả field đều là `final`, tức là **immutable** (tạo xong không sửa được). Muốn thay đổi thì tạo object mới. Nhờ vậy Riverpod phát hiện thay đổi đúng và UI không bị lỗi "sửa mà không render lại".

2.2 `data/models/content_block.dart`

```
/// One block of the detail content. `sealed` = the compiler knows
/// every subtype, so a `switch` over it must handle all 4 cases.
sealed class ContentBlock {
  const ContentBlock();
}

class ParagraphBlock extends ContentBlock {
  const ParagraphBlock(this.text);
  final String text;
}

class BulletListBlock extends ContentBlock {
  const BulletListBlock(this.items);
  final List<String> items;
}

class ImageBlock extends ContentBlock {
  const ImageBlock(this.url);
  final String url;
}
```

```
class HeadingBlock extends ContentBlock {
  const HeadingBlock(this.text);
  final String text;
}
```

Điểm hay của `sealed`: ở Bước 8, nếu sếp thêm loại khối thứ 5 (ví dụ `VideoBlock`) mà quên vẽ nó, compiler sẽ báo lỗi ngay. Không phải đến lúc chạy app mới phát hiện.

2.3 `data/models/service_item.dart`

```
import 'content_block.dart';

class ServiceItem {
  const ServiceItem({
    required this.id,
    required this.categoryId,
    required this.title,
    required this.thumbnailUrl,
    required this.bannerUrl,
    required this.headline,
    this.isHot = false,
    this.content = const [],
  });

  final String id;
  final String categoryId;
  final String title; // short title shown on the card (max 3 lines)
  final String thumbnailUrl;
  final String bannerUrl;
  final String headline; // long title shown in the floating card on detail
page
  final bool isHot;
  final List<ContentBlock> content;
}
```

2.4 `data/mock_data.dart`

```
import 'models/content_block.dart';
import 'models/service_category.dart';
import 'models/service_item.dart';

String _img(String seed, {int w = 600, int h = 400}) =>
    'https://picsum.photos/seed/$seed/$w/$h';

const String hotCategoryId = 'hot';
```

```
final List<ServiceCategory> mockCategories = [
  ServiceCategory(id: hotCategoryId, name: 'Dịch vụ hot', iconUrl: '', isHot:
true),
  ServiceCategory(id: 'skin', name: 'Điều trị da công nghệ cao', iconUrl:
_img('skin', w: 120, h: 120)),
  ServiceCategory(id: 'rejuv', name: 'Trẻ hoá da toàn diện', iconUrl:
_img('rejuv', w: 120, h: 120)),
  ServiceCategory(id: 'brow', name: 'Phun thêu thẩm mỹ', iconUrl:
_img('brow', w: 120, h: 120)),
  ServiceCategory(id: 'laser', name: 'Laser', iconUrl: _img('laser', w: 120,
h: 120)),
];

const String _lorem =
    'Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
'
    'tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
veniam, '
    'quis nostrud exercitation ullamco laboris nisi ut aliquip ex';

List<ContentBlock> _sampleContent(int i) => [
      const ParagraphBlock(_lorem),
      const BulletListBlock([
        'Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do
eiusmod tempor incididunt u',
        'Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do
eiusmod tempor incididunt u',
      ]),
      ImageBlock(_img('promo$i', w: 800, h: 400)),
      const HeadingBlock('Lorem ipsum dolor sit amet, consectetur'),
      const ParagraphBlock(_lorem),
      const BulletListBlock([
        'Lorem ipsum dolor sit amet, consectetur adipiscing elit',
        'Sed do eiusmod tempor incididunt ut labore et dolore',
      ]),
      const ParagraphBlock(_lorem),
    ];

final List<ServiceItem> mockServices = List.generate(12, (i) {
  final categories = ['skin', 'rejuv', 'brow', 'laser'];
  return ServiceItem(
    id: 'svc-$i',
    categoryId: categories[i % categories.length],
    isHot: i < 6, // first 6 items show up in "Dich vu hot"
    title: 'Sốc! chỉ 599K mà làm được trọn bộ trẻ hoá - sáng da?',
```

```
    thumbnailUrl: _img('thumb$i'),
    bannerUrl: _img('banner$i', w: 800, h: 520),
    headline: 'Lorem ipsum dolor sit amet, consectetur adipiscing elit, '
        'sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.',
    content: _sampleContent(i),
  );
});
```

- `List.generate(12, (i) => ...)` tạo 12 dịch vụ mà không phải viết tay từng cái.
- `picsum.photos/seed/xxx` luôn trả về cùng một ảnh cho cùng một `seed`, nên mỗi lần reload ảnh không bị đổi.

2.5 `data/service_repository.dart`

Repository là **lớp trung gian giữa UI và nguồn dữ liệu**. UI chỉ biết "gọi `getServices()` thì nhận được danh sách", còn dữ liệu lấy từ mock hay API thì UI không cần quan tâm. Sau này có backend thật, sếp viết thêm `ApiServiceRepository` và đổi 1 dòng ở provider. Không widget nào phải sửa.

```
import 'mock_data.dart';
import 'models/service_category.dart';
import 'models/service_item.dart';

abstract interface class ServiceRepository {
  Future<List<ServiceCategory>> getCategories();
  Future<List<ServiceItem>> getServices(String categoryId);
  Future<ServiceItem> getServiceById(String id);
}

class MockServiceRepository implements ServiceRepository {
  // Simulate network latency so loading states are visible
  static const Duration _delay = Duration(milliseconds: 600);

  @override
  Future<List<ServiceCategory>> getCategories() async {
    await Future<void>.delayed(_delay);
    return mockCategories;
  }

  @override
  Future<List<ServiceItem>> getServices(String categoryId) async {
    await Future<void>.delayed(_delay);
    if (categoryId == hotCategoryId) {
      return mockServices.where((s) => s.isHot).toList();
    }
```

```
    return mockServices.where((s) => s.categoryId == categoryId).toList();
  }

  @override
  Future<ServiceItem> getServiceById(String id) async {
    await Future<void>.delayed(_delay);
    return mockServices.firstWhere(
      (s) => s.id == id,
      orElse: () => throw Exception('Service $id not found'),
    );
  }
}
```

- `Future.delayed(600ms)` cố ý làm chậm để sếp thấy được trạng thái loading. Nếu mock trả về ngay thì sẽ không bao giờ test được spinner.
- `abstract interface class`: chỉ khai báo "hợp đồng" (các hàm phải có), giống `interface` trong TypeScript.

Bước 3: State với Riverpod

Mục tiêu: có 5 provider: repository, danh sách danh mục, danh mục đang chọn, danh sách dịch vụ theo danh mục và chi tiết 1 dịch vụ.

Giải thích: Provider trong Riverpod là "một ô chứa state toàn cục". Widget nào cần thì `ref.watch` ô đó, và khi giá trị trong ô đổi thì widget tự vẽ lại. So với bên web mà sếp quen:

Riverpod	Web tương đương	Dùng ở đây cho
`Provider`	Dependency injection, giống `@Injectable` trong NestJS	Repository
`NotifierProvider`	Zustand store	Danh mục đang chọn
`FutureProvider`	`useQuery` của React Query	Danh sách danh mục, danh sách dịch vụ
`FutureProvider.family`	`useQuery(['service', id])`	Chi tiết theo `id`

Luồng hoạt động khi người dùng bấm vào một danh mục:

```
flowchart LR
  A[Tap CategoryItem] --> B[selectedCategoryProvider<br/>= 'skin']
  B --> C[servicesProvider<br/>tự fetch lại]
  C --> D[Grid hiển thị<br/>loading rồi data]
```

`servicesProvider` **watch** `selectedCategoryProvider`, nên khi danh mục đổi thì nó tự gọi lại API. Sếp không cần viết `useEffect` hay gọi tay `refetch`.

3.1 `providers/service_providers.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/mock_data.dart';
import '../data/models/service_category.dart';
import '../data/models/service_item.dart';
import '../data/service_repository.dart';

/// Swap MockServiceRepository for ApiServiceRepository later — only this
line changes.
final serviceRepositoryProvider = Provider<ServiceRepository>(
  (ref) => MockServiceRepository(),
);

/// Horizontal category bar data.
final categoriesProvider = FutureProvider<List<ServiceCategory>>((ref) {
  return ref.watch(serviceRepositoryProvider).getCategories();
});

/// Which category is currently selected. Defaults to "hot".
class SelectedCategoryNotifier extends Notifier<String> {
  @override
  String build() => hotCategoryId;

  void select(String categoryId) => state = categoryId;
}

final selectedCategoryProvider =
    NotifierProvider<SelectedCategoryNotifier, String>
(SelectedCategoryNotifier.new);

/// Services of the selected category. Re-fetches automatically when
selection changes.
final servicesProvider = FutureProvider<List<ServiceItem>>((ref) {
  final categoryId = ref.watch(selectedCategoryProvider);
  return ref.watch(serviceRepositoryProvider).getServices(categoryId);
});

/// Detail of one service, keyed by id.
```

```
final serviceDetailProvider = FutureProvider.family<ServiceItem, String>
((ref, id) {
  return ref.watch(serviceRepositoryProvider).getServiceById(id);
});
```

3.2 Những điểm cần hiểu

- **`ref.watch` và `ref.read`:**
  - `ref.watch(x)` dùng trong `build` hoặc bên trong provider. Khi `x` đổi thì nơi gọi sẽ chạy lại.
  - `ref.read(x)` dùng trong callback như `onTap`, chỉ đọc một lần. **Không bao giờ** dùng `read` trong `build`.
- **`AsyncValue`:** `ref.watch(servicesProvider)` không trả về `List` mà trả về `AsyncValue<List>`, có 3 trạng thái `loading`, `data`, `error`. UI xử lý bằng `.when(...)`, giống `isLoading / data / error` của React Query.
- **`SelectedCategoryNotifier.new`:** đây là tear-off của constructor, tức là viết tắt cho `() => SelectedCategoryNotifier()`.
- **Cache:** mỗi `id` của `.family` được cache riêng. Mở lại dịch vụ `svc-1` lần thứ hai sẽ không phải fetch lại.

Tự kiểm tra: chạy `flutter analyze` và không có lỗi. Provider sẽ được dùng thật từ Bước 5.

Bước 4: Routing, `main.dart` và `app.dart`

Mục tiêu: app mở thẳng vào trang "Dịch vụ", bấm vào một item thì sang trang chi tiết `/services/svc-1`, bấm back thì quay lại.

Giải thích: `go_router` cho phép điều hướng bằng URL, giống Next.js. Route chi tiết được lồng *bên trong* route danh sách, nhờ đó khi mở `/services/svc-1` thì nút back tự biết quay về `/services`. Cách này cũng cần cho deep link sau này, ví dụ mở app từ thông báo khuyến mãi.

4.1 Tạo 2 trang tạm

Router cần import 2 trang, nên sếp tạo bản tạm trước, tới Bước 5 và Bước 7 sẽ viết lại:

```
// presentation/pages/service_list_page.dart
import 'package:flutter/material.dart';

class ServiceListPage extends StatelessWidget {
  const ServiceListPage({super.key});

  @override
  Widget build(BuildContext context) =>
      const Scaffold(body: Center(child: Text('List page')));
}
```

```
// presentation/pages/service_detail_page.dart
import 'package:flutter/material.dart';

class ServiceDetailPage extends StatelessWidget {
  const ServiceDetailPage({super.key, required this.serviceId});

  final String serviceId;

  @override
  Widget build(BuildContext context) =>
      Scaffold(body: Center(child: Text('Detail: $serviceId')));
}
```

4.2 `lib/core/router/app_router.dart`

```
import 'package:go_router/go_router.dart';

import '../../features/services/presentation/pages/service_detail_page.dart';
import '../../features/services/presentation/pages/service_list_page.dart';

abstract final class AppRoutes {
  static const String services = '/services';
  static String serviceDetail(String id) => '/services/$id';
}

final GoRouter appRouter = GoRouter(
  initialLocation: AppRoutes.services,
  routes: [
    GoRoute(
      path: AppRoutes.services,
      builder: (context, state) => const ServiceListPage(),
      routes: [
        // Nested: /services/:id — back button returns to /services
        GoRoute(
```

```
          path: ':id',
          builder: (context, state) =>
              ServiceDetailPage(serviceId: state.pathParameters['id']!),
        ),
      ],
    ),
  ],
);
```

- `':id'` là tham số động, giống `[id]` trong Next.js. Giá trị được lấy ra bằng `state.pathParameters['id']`.
- `AppRoutes` gom tất cả đường dẫn về một chỗ. Nhờ vậy sếp không phải gõ chuỗi `'/services/...'` rải rác và dễ gõ sai.

4.3 `lib/app.dart`

```
import 'package:flutter/material.dart';

import 'core/router/app_router.dart';
import 'core/theme/app_theme.dart';

class App extends StatelessWidget {
  const App({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'Ngoc Dung',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      routerConfig: appRouter,
    );
  }
}
```

4.4 `lib/main.dart` (xóa hết code cũ, thay bằng)

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app.dart';

void main() {
  // ProviderScope stores the state of every provider — must wrap the whole
app
  runApp(const ProviderScope(child: App()));
}
```

Thiếu `ProviderScope` là lỗi hay gặp nhất khi mới dùng Riverpod. App sẽ crash với thông báo *"No ProviderScope found"*.

4.5 Cách điều hướng (dùng từ Bước 6)

Muốn	Code
Mở trang chi tiết	`context.push(AppRoutes.serviceDetail(item.id))`
Quay lại	`context.pop()`
Kiểm tra có trang để quay lại không	`context.canPop()`

`push` thêm trang mới lên trên và có nút back. `go` thì thay thế cả stack. Chi tiết dịch vụ nên dùng `push`.

Tự kiểm tra: khởi động lại app (bấm `R` hoặc chạy lại `flutter run`). Màn hình phải hiện chữ "List page".

Bước 5: Màn danh sách, header và thanh danh mục

Mục tiêu: có header "Dịch vụ" và một khối trắng bo góc chứa danh mục cuộn ngang. Bấm vào danh mục nào thì danh mục đó được viền vàng và chữ chuyển sang màu vàng.

5.1 Phân tích bố cục

```
Scaffold
├── AppBar (white, back arrow + "Dịch vụ")
└── body: CustomScrollView           ← the whole page scrolls as one
    ├── SliverToBoxAdapter
    │   └── CategoryBar              ← white rounded box
    │       └── ListView horizontal
    │           └── CategoryItem × N  ← circle icon + 2-line label
    └── SliverGrid                   ← Step 6
```

Tại sao dùng `CustomScrollView` + sliver? Trong thiết kế, khi cuộn lưới xuống thì thanh danh mục cũng trôi lên theo. Nếu đặt `Column` chứa `ListView` và `GridView` thì sẽ thành 2 vùng cuộn lồng nhau và bị lỗi *unbounded height*. Sliver là các "mảnh" cùng nằm trong **một** vùng cuộn duy nhất:

- `SliverToBoxAdapter`: bọc một widget thường (box) để đặt được vào `CustomScrollView`.
- `SliverGrid` và `SliverList`: phiên bản sliver của Grid và List. Chỉ item nào hiển thị trên màn hình mới được vẽ.

5.2 `presentation/widgets/category_item.dart`

Một danh mục gồm icon tròn 60px và nhãn tối đa 2 dòng. Widget này là **StatelessWidget "ngốc"**: nó không biết gì về Riverpod, chỉ nhận dữ liệu và callback. Nhờ vậy dễ tái sử dụng và dễ test.

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';

import '../../../../core/theme/app_colors.dart';
import '../../../../core/theme/app_text_styles.dart';
import '../../data/models/service_category.dart';

class CategoryItem extends StatelessWidget {
  const CategoryItem({
    super.key,
    required this.category,
    required this.isSelected,
    required this.onTap,
  });

  final ServiceCategory category;
  final bool isSelected;
  final VoidCallback onTap;
```

```
  @override
  Widget build(BuildContext context) {
    final Color labelColor = isSelected ? AppColors.gold :
AppColors.textPrimary;

    return GestureDetector(
      onTap: onTap,
      behavior: HitTestBehavior.opaque, // tap on empty space inside also
counts
      child: SizedBox(
        width: 84,
        child: Column(
          children: [
            AnimatedContainer(
              duration: const Duration(milliseconds: 200),
              width: 60,
              height: 60,
              padding: const EdgeInsets.all(3),
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                border: Border.all(
                  color: isSelected ? AppColors.gold : Colors.transparent,
                  width: 2,
                ),
              ),
              child: ClipOval(child: _buildIcon()),
            ),
            const SizedBox(height: 8),
            Text(
              category.name,
              textAlign: TextAlign.center,
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: AppTextStyles.categoryLabel.copyWith(
                color: labelColor,
                fontWeight: isSelected ? FontWeight.w600 : FontWeight.w400,
              ),
            ),
          ],
        ),
      ),
    );
  }

  Widget _buildIcon() {
```

```
    if (category.isHot) {
      return ColoredBox(
        color: AppColors.gold.withValues(alpha: 0.15),
        child: const Icon(Icons.local_fire_department, color: AppColors.gold,
size: 32),
      );
    }
    return CachedNetworkImage(imageUrl: category.iconUrl, fit: BoxFit.cover);
  }
}
```

Những điểm cần hiểu:

- `AnimatedContainer`: khi `isSelected` đổi, viền tự chuyển màu mượt trong 200ms mà không cần viết `AnimationController`.
- `padding: 3` + `ClipOval`: tạo khoảng hở giữa viền vàng và ảnh, giống thiết kế.
- `copyWith`: lấy style gốc rồi chỉ đổi vài thuộc tính, không phải tạo `TextStyle` mới từ đầu.
- `SizedBox(width: 84)`: cố định bề ngang để nhãn dài như "Điều trị da công nghệ cao" tự xuống dòng.

5.3 `presentation/widgets/category_bar.dart`

Widget này là `ConsumerWidget` vì cần đọc provider. Nó nối state vào các `CategoryItem` "ngốc" bên trong.

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/theme/app_colors.dart';
import '../../providers/service_providers.dart';
import 'category_item.dart';

class CategoryBar extends ConsumerWidget {
  const CategoryBar({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final categoriesAsync = ref.watch(categoriesProvider);
    final selectedId = ref.watch(selectedCategoryProvider);

    return Container(
      height: 136,
      decoration: BoxDecoration(
        color: AppColors.surface,
```

```
        borderRadius: BorderRadius.circular(16),
      ),
      child: categoriesAsync.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (error, _) => const Center(child: Text('Không tải được danh
mục')),
        data: (categories) => ListView.separated(
          scrollDirection: Axis.horizontal,
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 16),
          itemCount: categories.length,
          separatorBuilder: (_, _) => const SizedBox(width: 4),
          itemBuilder: (context, index) {
            final category = categories[index];
            return CategoryItem(
              category: category,
              isSelected: category.id == selectedId,
              onTap: () =>
ref.read(selectedCategoryProvider.notifier).select(category.id),
            );
          },
        ),
      ),
    );
  }
}
```

Những điểm cần hiểu:

- `ListView` nằm ngang thì **phải có chiều cao cố định** (`height: 136`). Thiếu thì bị lỗi *"Horizontal viewport was given unbounded height"*.
- `onTap` dùng `ref.read(...notifier)` vì đây là callback. Nhắc lại: `watch` để vẽ, `read` để hành động.
- `.when(loading, error, data)` bắt sếp xử lý đủ cả 3 trạng thái. Đây là best practice: không để màn hình trắng khi có lỗi.

5.4 Viết lại `presentation/pages/service_list_page.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../widgets/category_bar.dart';

class ServiceListPage extends StatelessWidget {
  const ServiceListPage({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: IconButton(
          icon: const Icon(Icons.arrow_back_ios_new, size: 20),
          onPressed: () {
            if (context.canPop()) context.pop();
          },
        ),
        title: const Text('Dịch vụ'),
      ),
      body: const CustomScrollView(
        slivers: [
          SliverPadding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 12),
            sliver: SliverToBoxAdapter(child: CategoryBar()),
          ),
          // Step 6: the services grid goes here
        ],
      ),
    );
  }
}
```

Tự kiểm tra:

- [ ] Thấy header trắng, chữ "Dịch vụ" màu xanh năm giữa.
- [ ] Thấy spinner khoảng 0,6 giây, sau đó hiện 5 danh mục.
- [ ] "Dịch vụ hot" có viền vàng ngay từ đầu. Bấm sang danh mục khác thì viền vàng chuyển theo.
- [ ] Vuốt ngang được thanh danh mục.

Bước 6: ServiceCard và lưới 2 cột

Mục tiêu: bên dưới thanh danh mục là lưới 2 cột. Mỗi card có ảnh bo góc và tiêu đề tối đa 3 dòng. Đổi danh mục thì lưới tải lại, bấm vào card thì sang trang chi tiết. Xong bước này là **màn 1 hoàn chỉnh**.

6.1 Phân tích một card

```
Material (white, radius 12)       ← card background + clips the ripple
└── InkWell (onTap)                ← ripple effect on tap
    └── Padding 4
        └── Column
            ├── ClipRRect radius 8
            │   └── AspectRatio 16/10
            │       └── CachedNetworkImage
            └── Text (max 3 lines, …)
```

Trong thiết kế, ảnh bo góc nhỏ hơn card và cách mép card khoảng 4px. Vì vậy có 2 lớp bo góc: card `12` và ảnh `8`.

6.2 `presentation/widgets/service_card.dart`

```dart
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';

import '../../../../core/theme/app_colors.dart';
import '../../../../core/theme/app_text_styles.dart';
import '../../data/models/service_item.dart';

class ServiceCard extends StatelessWidget {
  const ServiceCard({super.key, required this.item, required this.onTap});

  final ServiceItem item;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: AppColors.surface,
      borderRadius: BorderRadius.circular(12),
      clipBehavior: Clip.antiAlias, // keep the ripple inside rounded corners
      child: InkWell(
        onTap: onTap,
        child: Padding(
          padding: const EdgeInsets.all(4),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              ClipRRect(
                borderRadius: BorderRadius.circular(8),
                child: AspectRatio(
                  aspectRatio: 16 / 10,
```

```
                  child: CachedNetworkImage(
                    imageUrl: item.thumbnailUrl,
                    fit: BoxFit.cover,
                    placeholder: (_, _) => const ColoredBox(color:
AppColors.background),
                    errorWidget: (_, _, _) => const
Icon(Icons.broken_image_outlined),
                  ),
                ),
              ),
              const SizedBox(height: 8),
              Padding(
                padding: const EdgeInsets.symmetric(horizontal: 4),
                child: Text(
                  item.title,
                  maxLines: 3,
                  overflow: TextOverflow.ellipsis,
                  style: AppTextStyles.cardTitle,
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Những điểm cần hiểu:

- **`Material` + `InkWell` thay cho `Container` + `GestureDetector`**: cách này có hiệu ứng gợn sóng khi bấm, người dùng biết là mình đã bấm trúng. `InkWell` cần có `Material` ở phía trên mới vẽ được hiệu ứng.
- **`AspectRatio`**: ảnh luôn giữ tỉ lệ 16:10 dù màn hình rộng hay hẹp. Không cần cố định `height` bằng pixel.
- **`placeholder` và `errorWidget`**: trong lúc ảnh chưa tải xong thì hiện ô màu nền, tải lỗi thì hiện icon. Tránh được lỗi giao diện bị "giật" khi ảnh hiện ra.
- **`(_, _)`**: Dart 3.7 trở lên cho phép dùng `_` cho nhiều tham số không dùng tới.

6.3 Thêm lưới vào `service_list_page.dart`

Trang cần đọc `servicesProvider`, nên đổi từ `StatelessWidget` sang `ConsumerWidget`:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../core/router/app_router.dart';
import '../../providers/service_providers.dart';
import '../widgets/category_bar.dart';
import '../widgets/service_card.dart';

class ServiceListPage extends ConsumerWidget {
  const ServiceListPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final servicesAsync = ref.watch(servicesProvider);

    return Scaffold(
      appBar: AppBar(
        leading: IconButton(
          icon: const Icon(Icons.arrow_back_ios_new, size: 20),
          onPressed: () {
            if (context.canPop()) context.pop();
          },
        ),
        title: const Text('Dịch vụ'),
      ),
      body: CustomScrollView(
        slivers: [
          const SliverPadding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 12),
            sliver: SliverToBoxAdapter(child: CategoryBar()),
          ),
          servicesAsync.when(
            loading: () => const SliverFillRemaining(
              hasScrollBody: false,
              child: Center(child: CircularProgressIndicator()),
            ),
            error: (error, _) => SliverFillRemaining(
              hasScrollBody: false,
              child: Center(child: Text('Đã có lỗi: $error')),
            ),
            data: (services) {
              if (services.isEmpty) {
                return const SliverFillRemaining(
                  hasScrollBody: false,
                  child: Center(child: Text('Chưa có dịch vụ nào')),
```

```
                );
              }
              return SliverPadding(
                padding: const EdgeInsets.fromLTRB(16, 0, 16, 24),
                sliver: SliverGrid.builder(
                  gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
                    crossAxisCount: 2,
                    mainAxisSpacing: 12,
                    crossAxisSpacing: 12,
                    childAspectRatio: 0.86, // width / height of each card
                  ),
                  itemCount: services.length,
                  itemBuilder: (context, index) {
                    final item = services[index];
                    return ServiceCard(
                      item: item,
                      onTap: () =>
context.push(AppRoutes.serviceDetail(item.id)),
                    );
                  },
                ),
              );
            },
          ),
        ],
      ),
    );
  }
}
```

Những điểm cần hiểu:

- **`childAspectRatio: 0.86`**: ô lưới có chiều rộng / chiều cao = 0,86, tức là hơi cao hơn rộng. Em tính số này cho vừa ảnh 16:10 cộng 3 dòng chữ. Nếu thấy sọc vàng-đen báo *overflow* ở đáy card thì giảm số này xuống (0,8), còn thừa khoảng trắng nhiều thì tăng lên.
- **Trong `.when` luôn trả về sliver:** `CustomScrollView` chỉ nhận sliver, nên cả 3 nhánh (loading, error, data) đều phải trả về `SliverXxx`. Nếu trả về `Center` trực tiếp sẽ bị lỗi *"A RenderViewport expected a child of type RenderSliver"*.
- **`SliverFillRemaining(hasScrollBody: false)`**: chiếm hết phần còn lại của màn hình để spinner hoặc thông báo nằm giữa.
- **Trạng thái rỗng:** đây là trạng thái thứ 4 hay bị quên. Danh mục chưa có dịch vụ nào thì phải báo cho người dùng, không để màn hình trống.

Tự kiểm tra:

- [ ] Thấy 6 card "hot" xếp 2 cột, ảnh bo góc, tiêu đề 3 dòng.
- [ ] Cuộn xuống thì thanh danh mục trôi lên theo.
- [ ] Bấm "Laser" thì lưới hiện spinner rồi chỉ còn 3 card.
- [ ] Bấm vào card thì sang trang "Detail: svc-x", bấm back thì quay lại đúng danh mục đang chọn.

Bước 7: Màn chi tiết, header banner và card tiêu đề

Mục tiêu: banner tràn lên tận status bar, nút back tròn trắng ở góc trái, card tiêu đề trắng nằm đè lên mép dưới của banner. Đây là phần khó nhất về layout trong cả app.

7.1 Phân tích: tại sao là `Stack`

`Column` xếp các phần tử nối tiếp nhau, nên không thể cho phần tử này đè lên phần tử kia. `Stack` thì xếp **chồng lớp** theo trục Z: con đứng sau trong danh sách sẽ nằm trên. Header có 3 lớp:

```
Stack  (height = decided by the title card, NOT the banner)
├── [1] Positioned(top: 0, height: 240)  → banner image       (bottom layer)
├── [2] Padding(top: 240 - 32)           → white title card   (overlaps
banner by 32px)
└── [3] Positioned(top: safeTop + 8)     → round back button  (top layer)
```

Mẹo quan trọng: chiều cao của `Stack` bằng chiều cao của con **không** có `Positioned`, ở đây là lớp [2]. Card tiêu đề có thể dài 2, 3 hay 4 dòng, nên để nó quyết định chiều cao thì header luôn vừa khít. Nếu làm kiểu `Positioned(bottom: -70)` như nhiều tutorial, phần card nằm ngoài `Stack` sẽ **không bấm được** và dễ bị nội dung bên dưới che mất.

7.2 `presentation/widgets/detail_header.dart`

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../../../../core/theme/app_colors.dart';
import '../../../../core/theme/app_text_styles.dart';

class DetailHeader extends StatelessWidget {
  const DetailHeader({super.key, required this.bannerUrl, required
this.title});

  final String bannerUrl;
```

```
  final String title;

  static const double _overlap = 32; // how much the title card covers the
banner

  @override
  Widget build(BuildContext context) {
    final double safeTop = MediaQuery.paddingOf(context).top; // status bar
height
    final double bannerHeight = MediaQuery.sizeOf(context).width * 0.56 +
safeTop;

    return ColoredBox(
      color: AppColors.background, // light-blue band behind the card's lower
half
      child: Stack(
        children: [
          // [1] Banner — bottom layer
          Positioned(
            top: 0,
            left: 0,
            right: 0,
            height: bannerHeight,
            child: CachedNetworkImage(imageUrl: bannerUrl, fit:
BoxFit.cover),
          ),

          // [2] Title card — NOT positioned, so it decides the Stack's
height
          Padding(
            padding: EdgeInsets.fromLTRB(32, bannerHeight - _overlap, 32,
20),
            child: _TitleCard(title: title),
          ),

          // [3] Back button — top layer
          Positioned(
            top: safeTop + 8,
            left: 16,
            child: const _CircleBackButton(),
          ),
        ],
      ),
    );
  }
}
```

```
class _TitleCard extends StatelessWidget {
  const _TitleCard({required this.title});

  final String title;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      decoration: BoxDecoration(
        color: AppColors.surface,
        borderRadius: BorderRadius.circular(16),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: 0.06),
            blurRadius: 16,
            offset: const Offset(0, 4),
          ),
        ],
      ),
      child: Text(title, textAlign: TextAlign.center, style:
AppTextStyles.detailTitle),
    );
  }
}

class _CircleBackButton extends StatelessWidget {
  const _CircleBackButton();

  @override
  Widget build(BuildContext context) {
    return Material(
      color: AppColors.surface,
      shape: const CircleBorder(),
      elevation: 2,
      child: IconButton(
        icon: const Icon(Icons.chevron_left, color: AppColors.primary, size:
30),
        onPressed: () => context.pop(),
      ),
    );
  }
}
```

Những điểm cần hiểu:

- **`MediaQuery.paddingOf(context).top`**: chiều cao của status bar cùng tai thỏ hoặc Dynamic Island. Banner cộng thêm khoảng này để tràn lên tận đỉnh màn hình, còn nút back cũng cộng thêm để không bị che.
- **`width * 0.56`**: banner cao theo tỉ lệ với bề ngang màn hình, nên máy nhỏ hay máy lớn đều cân đối.
- **`sizeOf` / `paddingOf` thay cho `MediaQuery.of(context).size`**: widget chỉ rebuild khi đúng giá trị đó đổi. Đây là best practice về hiệu năng.
- **Widget private `_TitleCard`**: dấu `_` đầu tên nghĩa là chỉ dùng trong file này. Tách thành class riêng, không viết dạng hàm `_buildTitleCard()`, giúp Flutter tối ưu rebuild tốt hơn.

7.3 Viết lại `presentation/pages/service_detail_page.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/theme/app_colors.dart';
import '../../providers/service_providers.dart';
import '../widgets/detail_header.dart';

class ServiceDetailPage extends ConsumerWidget {
  const ServiceDetailPage({super.key, required this.serviceId});

  final String serviceId;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final serviceAsync = ref.watch(serviceDetailProvider(serviceId));

    // White status bar icons on top of the dark banner
    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: SystemUiOverlayStyle.light,
      child: Scaffold(
        backgroundColor: AppColors.surface,
        body: serviceAsync.when(
          loading: () => const Center(child: CircularProgressIndicator()),
          error: (error, _) => Center(child: Text('Đã có lỗi: $error')),
          data: (service) => CustomScrollView(
            slivers: [
              SliverToBoxAdapter(
                child: DetailHeader(bannerUrl: service.bannerUrl, title:
```

```
service.headline),
              ),
              // Step 8: content blocks go here
            ],
          ),
        ),
      ),
    );
  }
}
```

- **Không có `AppBar`:** thiết kế không có thanh tiêu đề ở màn này, banner chiếm luôn phần trên cùng. Không bọc `SafeArea` cũng là cố ý, để banner tràn lên được.
- **`AnnotatedRegion`:** đổi giờ, sóng, pin trên status bar sang màu trắng để đọc được trên nền banner màu xanh đậm.
- **`serviceDetailProvider(serviceId)`:** đây là cách gọi `.family`, truyền `id` vào giống như gọi hàm.

Tự kiểm tra:

- [ ] Bấm một card ở màn danh sách thì thấy spinner, sau đó banner hiện lên tràn tận status bar.
- [ ] Giờ và pin trên status bar có màu trắng.
- [ ] Card tiêu đề đè lên mép dưới banner, phần dưới card nằm trên dải nền xanh nhạt.
- [ ] Nút back tròn trắng bấm được và quay về danh sách.
- [ ] Thử sửa `headline` trong mock cho dài gấp đôi, card phải tự cao lên mà không bị lỗi.

Bước 8: Nội dung chi tiết

Mục tiêu: bên dưới header là nội dung giống thiết kế, gồm đoạn văn, danh sách bullet, ảnh khuyến mãi bo góc và tiêu đề phụ in đậm. Nội dung được vẽ từ `List<ContentBlock>` đã tạo ở Bước 2.

Giải thích: Thay vì viết cứng giao diện cho từng dịch vụ, sếp viết **một widget biết cách vẽ 1 khối**, rồi lặp qua danh sách khối. Backend muốn nội dung gì (đoạn văn trước hay ảnh trước, bao nhiêu đoạn) thì app tự hiển thị đúng, không cần phát hành bản app mới.

8.1 `presentation/widgets/content_block_view.dart`

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';

import '../../../../core/theme/app_text_styles.dart';
```

```
import '../../data/models/content_block.dart';

class ContentBlockView extends StatelessWidget {
  const ContentBlockView({super.key, required this.block});

  final ContentBlock block;

  @override
  Widget build(BuildContext context) {
    // Exhaustive switch: adding a new ContentBlock subtype without handling
it = compile error
    return switch (block) {
      ParagraphBlock(:final text) => Text(text, style: AppTextStyles.body),
      HeadingBlock(:final text) => Padding(
          padding: const EdgeInsets.only(top: 8),
          child: Text(text, style: AppTextStyles.sectionHeading),
        ),
      BulletListBlock(:final items) => Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [for (final item in items) _BulletItem(text: item)],
        ),
      ImageBlock(:final url) => ClipRRect(
          borderRadius: BorderRadius.circular(12),
          child: CachedNetworkImage(imageUrl: url, width: double.infinity,
fit: BoxFit.cover),
        ),
    };
  }
}

class _BulletItem extends StatelessWidget {
  const _BulletItem({required this.text});

  final String text;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(top: 4),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start, // bullet stays on the
first line
        children: [
          const Padding(
            padding: EdgeInsets.symmetric(horizontal: 8),
            child: Text('•', style: AppTextStyles.body),
```

```
            ),
            Expanded(child: Text(text, style: AppTextStyles.body)),
          ],
        ),
      );
    }
}
```

Những điểm cần hiểu:

- **`switch` expression + pattern (Dart 3):** `ParagraphBlock(:final text)` có nghĩa là "nếu `block` là `ParagraphBlock` thì lấy field `text` ra một biến tên `text`". Đây là destructuring, giống `const { text } = block` trong TypeScript nhưng có kèm kiểm tra kiểu.
- **Không cần `default`:** vì `ContentBlock` là `sealed`, compiler biết chắc chỉ có 4 loại nên không đòi nhánh `default`. Sếp thử xóa dòng `ImageBlock` sẽ thấy lỗi *"The type 'ContentBlock' is not exhaustively matched"*.
- **`[for (final item in items) ...]`:** collection-for, tạo list widget ngay trong list literal. Cách này gọn hơn `items.map(...).toList()`.
- **Bullet tự vẽ bằng `Row`:** `crossAxisAlignment.start` giữ dấu `•` ở dòng đầu, còn `Expanded` cho chữ tự xuống dòng và các dòng sau thẳng hàng với chữ ở dòng đầu. Thiết kế cũng thụt lề như vậy.

8.2 Thêm nội dung vào `service_detail_page.dart`

Thêm import `../widgets/content_block_view.dart`, sau đó thay dòng comment `// Step 8` bằng:

```
SliverPadding(
  // Bottom 120 leaves room for the floating booking button (Step 9)
  padding: const EdgeInsets.fromLTRB(16, 20, 16, 120),
  sliver: SliverList.separated(
    itemCount: service.content.length,
    itemBuilder: (context, index) => ContentBlockView(block:
service.content[index]),
    separatorBuilder: (_, _) => const SizedBox(height: 12),
  ),
),
```

- **`SliverList.separated`:** vẽ từng khối một cách lười (lazy), có khoảng cách 12px giữa các khối. Bài viết có 50 khối vẫn mượt.
- **`bottom: 120`:** để chừa chỗ cho nút "Đặt lịch" ở Bước 9. Nếu không, đoạn văn cuối sẽ bị nút che mất.

8.3 Nếu backend trả về HTML

Nhiều CMS như WordPress hay Strapi trả nội dung dưới dạng HTML chứ không phải JSON từng khối. Khi đó sếp có 2 cách:

Cách	Ưu điểm	Nhược điểm
Dùng package `flutter_html` để render thẳng	Nhanh, hỗ trợ mọi thẻ	Khó chỉnh đúng style thiết kế, nặng hơn
Nhờ backend trả JSON dạng khối như Bước 2	Giao diện chuẩn, kiểm soát được 100%	Backend phải làm thêm

Em khuyên dùng cách 2 nếu sếp có quyền quyết định API. Dù chọn cách nào, `ContentBlockView` vẫn là nơi duy nhất cần sửa.

Tự kiểm tra:

- [ ] Thứ tự hiển thị là đoạn văn, 2 bullet, ảnh bo góc, tiêu đề đậm, đoạn văn.
- [ ] Bullet xuống dòng thì dòng thứ 2 thẳng hàng với chữ, không thụt về dưới dấu `•`.
- [ ] Cuối trang có khoảng trống khoảng 120px.

Bước 9: Nút "Đặt lịch" ẩn/hiện khi cuộn

Mục tiêu: nút gradient xanh bo tròn nổi phía trên nội dung. Cuộn xuống để đọc thì nút trượt xuống và ẩn đi (frame 3 trong thiết kế). Cuộn ngược lên hoặc tới cuối trang thì nút hiện lại. Xong bước này là **màn 2 hoàn chỉnh**.

9.1 Phân tích

```
Scaffold
└── Stack
    ├── CustomScrollView (controller: _scrollController)   ← header + content
    └── Positioned(bottom)
        └── SafeArea                                        ← avoid the
iPhone home indicator
            └── AnimatedSlide (show → 0, hide → slide down)
                └── BookingButton (gradient pill)
```

- **Tại sao không dùng `Scaffold.bottomNavigationBar`?** Vì `bottomNavigationBar` chiếm hẳn một vùng riêng ở đáy, nội dung bị thu ngắn lại. Trong thiết kế, nút **nổi đè** lên nội dung, nên phải dùng `Stack`.

- **Muốn biết người dùng đang cuộn lên hay xuống:** cần `ScrollController`. Controller này phải được tạo trong `initState` và hủy trong `dispose`, nên trang phải chuyển sang **Stateful**, cụ thể là `ConsumerStatefulWidget` vì vẫn cần `ref`.

9.2 `presentation/widgets/booking_button.dart`

```
import 'package:flutter/material.dart';

import '../../../../core/theme/app_colors.dart';
import '../../../../core/theme/app_text_styles.dart';

class BookingButton extends StatelessWidget {
  const BookingButton({super.key, required this.onPressed});

  final VoidCallback onPressed;

  static final BorderRadius _radius = BorderRadius.circular(32);

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: BoxDecoration(
        gradient: AppColors.buttonGradient,
        borderRadius: _radius,
        boxShadow: [
          BoxShadow(
            color: AppColors.primary.withValues(alpha: 0.35),
            blurRadius: 16,
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: Material(
        type: MaterialType.transparency, // lets the gradient show through,
keeps the ripple
        child: InkWell(
          borderRadius: _radius,
          onTap: onPressed,
          child: const SizedBox(
            height: 60,
            width: double.infinity,
            child: Center(child: Text('Đặt lịch', style:
AppTextStyles.button)),
          ),
        ),
```

```
      ),
    );
  }
}
```

- **Tại sao không dùng `ElevatedButton`?** Button có sẵn của Material chỉ nhận màu nền đơn sắc, không nhận gradient. Vì vậy phải tự ghép: `DecoratedBox` vẽ gradient, `Material` trong suốt + `InkWell` tạo hiệu ứng khi bấm.
- **`boxShadow` cùng tông màu xanh:** làm nút trông nổi hẳn lên khỏi nền trắng, đúng với thiết kế.

9.3 Viết lại `service_detail_page.dart` (bản cuối)

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/theme/app_colors.dart';
import '../../data/models/service_item.dart';
import '../../providers/service_providers.dart';
import '../widgets/booking_button.dart';
import '../widgets/content_block_view.dart';
import '../widgets/detail_header.dart';

class ServiceDetailPage extends ConsumerStatefulWidget {
  const ServiceDetailPage({super.key, required this.serviceId});

  final String serviceId;

  @override
  ConsumerState<ServiceDetailPage> createState() =>
_ServiceDetailPageState();
}

class _ServiceDetailPageState extends ConsumerState<ServiceDetailPage> {
  final ScrollController _scrollController = ScrollController();
  bool _showButton = true;

  @override
  void initState() {
    super.initState();
    _scrollController.addListener(_onScroll);
  }

  @override
```

```
  void dispose() {
    _scrollController.dispose(); // always dispose controllers to avoid
memory leaks
    super.dispose();
  }

  void _onScroll() {
    final position = _scrollController.position;
    final bool isAtBottom = position.pixels >= position.maxScrollExtent - 24;

    // null = no change (finger lifted / idle) -> keep current state
    final bool? shouldShow = switch (position.userScrollDirection) {
      _ when isAtBottom => true,
      ScrollDirection.reverse => false, // content moving up = user scrolling
down
      ScrollDirection.forward => true,
      ScrollDirection.idle => null,
    };

    if (shouldShow == null || shouldShow == _showButton) return;
    setState(() => _showButton = shouldShow);
  }

  void _onBook(ServiceItem service) {
    // TODO: navigate to the booking flow
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Đặt lịch: ${service.title}')),
    );
  }

  @override
  Widget build(BuildContext context) {
    final serviceAsync = ref.watch(serviceDetailProvider(widget.serviceId));

    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: SystemUiOverlayStyle.light,
      child: Scaffold(
        backgroundColor: AppColors.surface,
        body: serviceAsync.when(
          loading: () => const Center(child: CircularProgressIndicator()),
          error: (error, _) => Center(child: Text('Đã có lỗi: $error')),
          data: (service) => Stack(
            children: [
              CustomScrollView(
                controller: _scrollController,
                slivers: [
```

```
                  SliverToBoxAdapter(
                    child: DetailHeader(bannerUrl: service.bannerUrl, title:
service.headline),
                  ),
                  SliverPadding(
                    padding: const EdgeInsets.fromLTRB(16, 20, 16, 120),
                    sliver: SliverList.separated(
                      itemCount: service.content.length,
                      itemBuilder: (context, index) =>
                          ContentBlockView(block: service.content[index]),
                      separatorBuilder: (_, _) => const SizedBox(height: 12),
                    ),
                  ),
                ],
              ),
              Positioned(
                left: 20,
                right: 20,
                bottom: 0,
                child: SafeArea(
                  top: false,
                  minimum: const EdgeInsets.only(bottom: 16),
                  child: IgnorePointer(
                    ignoring: !_showButton, // hidden button must not catch
taps
                    child: AnimatedSlide(
                      offset: _showButton ? Offset.zero : const Offset(0, 2),
                      duration: const Duration(milliseconds: 250),
                      curve: Curves.easeOut,
                      child: BookingButton(onPressed: () =>
_onBook(service)),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

9.4 Những điểm cần hiểu

- **Vòng đời của `State`:**

  - `initState` chạy 1 lần khi widget được tạo, nên gắn listener ở đây.
  - `dispose` chạy khi rời trang, nên hủy controller ở đây. Quên `dispose` sẽ gây rò rỉ bộ nhớ (memory leak).
  - Trong `ConsumerState`, `ref` dùng được trực tiếp và `serviceId` được lấy qua `widget.serviceId`.
- **`userScrollDirection` dễ gây nhầm:** `reverse` nghĩa là nội dung đang trôi **lên**, tức người dùng đang đọc **xuống**. `idle` là lúc người dùng đã thả tay. Gặp `idle` thì em giữ nguyên trạng thái, nếu không nút sẽ nháy lên mỗi lần dừng cuộn.
- **Chỉ gọi `setState` khi thật sự đổi:** `_onScroll` chạy khoảng 60 lần mỗi giây trong lúc cuộn. Dòng `shouldShow == _showButton` giúp tránh rebuild cả trang 60 lần mỗi giây.
- **`_ when isAtBottom`:** pattern với điều kiện (guard). Nhánh này được xét đầu tiên, nên ở cuối trang thì nút luôn hiện để người dùng đọc xong là đặt lịch được ngay.
- **`AnimatedSlide(offset: Offset(0, 2))`:** đẩy nút xuống 2 lần chiều cao của chính nó, tức là ra khỏi màn hình, có animation. `IgnorePointer` bảo đảm nút đang ẩn không bấm nhầm được.
- **`SafeArea(top: false)`:** chỉ chừa khoảng cho thanh home indicator ở đáy iPhone, không đụng vào phía trên.
- **`ScrollDirection` báo lỗi undefined?** Thêm `import 'package:flutter/rendering.dart';`.

Tự kiểm tra:

- [ ] Mở trang chi tiết thì thấy nút "Đặt lịch" gradient ở đáy, không dính vào home indicator.
- [ ] Vuốt đọc xuống thì nút trượt mượt xuống dưới và biến mất (giống frame 3).
- [ ] Vuốt ngược lên thì nút hiện lại. Cuộn tới đáy trang thì nút cũng hiện.
- [ ] Bấm nút thì hiện SnackBar.
- [ ] Thoát ra rồi vào lại nhiều lần, console không báo lỗi "used after being disposed".

Tự kiểm tra, lỗi thường gặp và bài tập nâng cao

Làm xong 10 bước, sếp đối chiếu với thiết kế theo checklist dưới đây. Nếu bị lỗi thì tra bảng "Lỗi thường gặp" trước.

Checklist cuối cùng

- [ ] `flutter analyze` không còn lỗi hay cảnh báo nào.
- [ ] Màn 1: header, thanh danh mục (viền vàng khi chọn), lưới 2 cột, đổi danh mục thì lưới tải lại.

- [ ] Màn 2: banner tràn lên status bar, nút back tròn, card tiêu đề đè lên banner, nội dung đủ 4 loại khối.
- [ ] Nút Đặt lịch ẩn khi cuộn xuống, hiện khi cuộn lên hoặc tới cuối trang.
- [ ] Chạy thử trên 2 kích thước màn hình (iPhone 18 Pro Max và một máy nhỏ như iPhone SE) không bị overflow.
- [ ] Vào Cài đặt, tăng cỡ chữ hệ thống lên lớn nhất, giao diện vẫn không vỡ.

Lỗi thường gặp

Thông báo lỗi / Hiện tượng	Nguyên nhân	Cách sửa
`No ProviderScope found`	Quên bọc app bằng `ProviderScope`	Xem Bước 4.4
`Horizontal viewport was given unbounded height`	`ListView` ngang không có chiều cao	Bọc bằng `SizedBox` hoặc `Container(height: ...)` (Bước 5.3)
`Vertical viewport was given unbounded height`	Đặt `ListView` / `GridView` trong `Column`	Dùng `CustomScrollView` + sliver (Bước 5.1)
`A RenderViewport expected a child of type RenderSliver`	Cho widget thường vào `slivers: [...]`	Bọc bằng `SliverToBoxAdapter`
Sọc vàng-đen `BOTTOM OVERFLOWED BY xx PIXELS` trong card	`childAspectRatio` quá lớn	Giảm xuống, ví dụ 0,86 → 0,8 (Bước 6.3)
Bấm danh mục nhưng không đổi gì	Dùng `ref.read` trong `build` thay vì `ref.watch`	`build` luôn dùng `watch`
`ScrollController was used after being disposed`	Gọi `dispose` sai thứ tự hoặc dùng chung controller	`_scrollController.dispose()` trước `super.dispose()`, mỗi trang một controller

Thông báo lỗi / Hiện tượng	Nguyên nhân	Cách sửa
Ảnh không hiện trên macOS	macOS chặn truy cập mạng mặc định	Thêm key `com.apple.security.network.client` = `true` vào `macos/Runner/DebugProfile.entitlements`
Status bar vẫn màu đen ở màn chi tiết	`AppBar` của màn khác ghi đè style	Kiểm tra `AnnotatedRegion` có bọc ngoài `Scaffold` (Bước 7.3)
Hot reload không thấy thay đổi	Sửa `main()`, provider, hoặc `initState`	Bấm `R` (hot restart) thay vì `r`

Bài tập nâng cao (theo độ khó tăng dần)

1. **Skeleton loading:** thay `CircularProgressIndicator` bằng các khối xám có hiệu ứng lấp lánh, dùng package `shimmer`. Gợi ý: tạo widget `ServiceCardSkeleton` có cùng kích thước với `ServiceCard`.
2. **Kéo để tải lại:** bọc `CustomScrollView` của màn 1 bằng `RefreshIndicator`. Trong `onRefresh` gọi `ref.refresh(servicesProvider.future)`.
3. **Hero animation:** khi bấm card, ảnh thumbnail "bay" lên thành banner. Gợi ý: bọc ảnh ở cả 2 nơi bằng `Hero(tag: 'service-${item.id}')`.
4. **Banner co giãn khi kéo quá đầu trang:** thay header bằng `SliverAppBar(expandedHeight: ..., stretch: true, flexibleSpace: FlexibleSpaceBar(...))`.
5. **Gọi API thật:** thêm `dio`, viết `ApiServiceRepository implements ServiceRepository` có hàm `fromJson` cho từng model, rồi đổi đúng 1 dòng trong `serviceRepositoryProvider`. Sếp sẽ thấy không widget nào phải sửa. Đó là lợi ích của kiến trúc ở Bước 2.
6. **Viết test:** widget test cho `CategoryItem`, kiểm tra khi `isSelected: true` thì chữ có màu `AppColors.gold`. Dùng `ProviderScope(overrides: [...])` để thay repository bằng bản giả khi test cả trang.

Làm xong bước nào mà muốn em review code, sếp cứ dán code hoặc để comment ngay tại bước đó trong doc này.